ISSUE2PROOF · COMPLETE GUIDE

One project. Three levels of explanation.

From “a toy remembers how it broke” to a professional architecture for executable regression memory.

“Regression memory for AI coding agents that survives the agent.”

Solo project · Nebius × NVIDIA Global AI Hackathon · October 2026

How to use this guide

Start simple. Add detail only when ready.

The three explanations describe the same product. Each layer adds vocabulary, mechanisms, and evidence.

1	**Like talking to a two-year-old** A picture-story about remembering a broken toy.	3
2	**Like talking to a new graduate** The software problem, workflow, worked example, and integrations.	5
3	**Professional explanation** Architecture, AI routing, evidence model, security, evaluation, and limits.	10
4	**How to explain it in 30 seconds** A judge-ready pitch and precise claims.	17

The central idea
A normal coding agent can propose and test a change. Issue2Proof turns a previously fixed failure into durable, executable evidence, finds that evidence when related code changes, and replays it independently before the new change ships.

The little toy that learned

■

You have a toy car.
One day, a wheel falls off.

■■■

A helper fixes it.
The car can roll again.

■

The toy keeps a tiny memory.
“My wheel must stay on when I roll.”

■

Next time somebody changes the car,
the toy checks its wheel before going outside.

Now give each part its real name

■ **The toy** The software project.	■ **The fallen wheel** A bug that users found.
■ **The helper** A person or AI coding agent.	■ **The tiny memory** A **Memory Capsule**.
■ **The wheel check** A frozen regression test.	■■ **The safe playroom** A fresh Nebius Sandbox.

Issue2Proof remembers how software broke, so the same kind of break can be checked again later.

That is the whole project. The remaining pages explain how this happens and why it is different from an ordinary coding assistant.

The problem: tests remember behavior, but often lose the story

A bug gets fixed today. Six months later, another person or agent refactors nearby code and accidentally brings the bug back.

Ordinary project memory

- Git commit and pull request
- Comments and rule files
- A test somewhere in the suite
- Possibly an agent memory note

VS

Issue2Proof memory

- Precise behavioral claim
- Why and where it applies
- Frozen test plus SHA-256 hash
- Execution provenance and boundaries

The key contrast is precise: useful prose and memory features already exist in Claude Code, Cursor, IDEs, and repositories. Issue2Proof adds executable, integrity-checked evidence that can be recalled and replayed across those tools.

What “regression” means

A regression happens when behavior that used to work becomes broken after a later change. A regression test is a small program that checks the specific behavior that previously failed.

What “survives the agent” means

The evidence lives in a service or portable bundle rather than depending on one model’s chat history. A different agent, IDE, CI job, or future session can retrieve the same contract.

How a failure becomes durable memory

1

Import and describe

Give Issue2Proof a public GitHub repository or bounded ZIP and describe the failure and expected behavior.

2

Reproduce first

Nemotron proposes a focused regression. The original revision runs in an isolated Nebius Sandbox. The test must demonstrate the failure before any repair is trusted.

3

Try a repair

The engineering model proposes a bounded code change. Ordinary tests, the new regression, and withheld independent checks execute in clean sandboxes.

4

Accept only observed success

A repair becomes `checks_passed` only after the recorded gates pass. Failures and uncertainty remain visible.

5

Create a Memory Capsule

The behavioral claim, paths, assumptions, hashes, environment, operation IDs, and frozen regression become a versioned record.

6

Recall and replay later

When related code changes, Issue2Proof finds the capsule and replays its frozen behavior on the proposed revision.

Failure reported → **Evidence** executed → **Capsule** stored → **Future PR** protected

Requests and the Unicode length bug

The old failure

A request body containing non-ASCII text reported its number of characters instead of its number of UTF-8 bytes. For example, a short string could occupy more bytes than characters, producing the wrong `Content-Length`.

1 · Behavioral claim

Unicode string bodies must use encoded byte length while ASCII, bytes, and partially consumed streams keep working.

2 · Frozen check

A regression test exercises the exact behavior. Its source receives a SHA-256 hash.

3 · Future change

An agent later “simplifies” `super_len` and removes the encoding case because visible tests still pass.

4 · Memory replay

The capsule is recalled. Its frozen test fails on the proposed revision, so the audit returns `memory_conflict`.

Why plain CI can miss this: a project’s visible suite may not include the exact historical boundary. Issue2Proof keeps the known failure as an explicit contract and routes it to relevant future changes.

This is one of the repository’s two public upstream demonstration histories. It is disclosed as a known case, not treated as a held-out benchmark.

Is this just another coding assistant?

No. It is a memory and verification layer that works with coding assistants.

Capability	Typical coding assistant	Issue2Proof
Write or edit code	Primary job	Can investigate and propose a repair, but future audits do not generate a patch
Run current tests	Often, with permission	Runs recorded gates in fresh isolated sandboxes
Remember guidance	Rules, notes, chat memory	Versioned behavioral claims tied to executable evidence
Cross-agent portability	Varies by product	REST, MCP, CLI, GitHub Action, and portable bundles
Historical provenance	Usually inferred from repository context	Hashes, operation IDs, image, dependencies, assumptions, and boundaries
Independent replay	The editing agent may run its own checks	A protected service can retain the regression and audit the agent's revision

Important: Issue2Proof does not claim that existing agents lack memory or testing. Its contribution is turning an observed repair into structured, replayable evidence that remains available to many agents and CI systems.

The browser is only one doorway

Web application

Import a GitHub repository or ZIP, review an investigation, create a capsule, search memory, and inspect audit evidence.

human review

MCP tools

`recall_repository_memory` runs before editing. `audit_repository_revision` runs after editing in the same agent session.

Claude Code

Cursor

Codex

GitHub pull request gate

The composite action reads pull request text and changed paths, recalls a portable bundle for free, and can request full remote replay.

CI

merge gate

REST and CLI

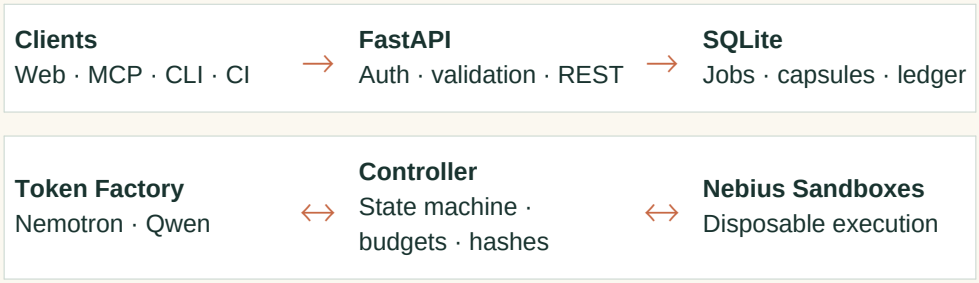
Any IDE extension or internal tool can request a model-neutral context packet and start an authenticated audit.

vendor-neutral

```
coding agent
→ recall_repository_memory("Refactor safe_join")
→ edit code
→ audit_repository_revision(".", memory_ids)
→ memory_preserved | memory_conflict | memory_uncertain
```

No ZIP is required for a public GitHub repository. ZIP remains available for private or offline source. Local agent and CI integrations package the current working revision automatically.

System architecture



Control plane

FastAPI accepts bounded inputs. SQLite persists investigation reports, queue leases, checkpoints, Memory Capsules, cached embeddings, cancellations, and cost reservations. A durable worker claims jobs and resumes only from recorded checkpoints.

Inference plane

Nebius Token Factory supplies role-specific models. Responses must satisfy narrow JSON schemas. Claims produced by a model are treated as proposals until execution provides evidence.

Execution plane

Fresh Nebius Sandboxes execute repository and model-generated code. Dependency preparation is separated from offline test execution. Operation IDs, immutable image identifiers, outputs, and hashes return to the controller.

Integration plane

MCP, REST, CLI, a composite GitHub Action, and portable bundles expose the same repository-memory concept without tying it to one IDE or model vendor.

What is inside a Memory Capsule?

issue2proof.memory.v1	
Identity ID, repository key, version, creation time, supersession link	**Claim** The behavior that future changes must preserve
Scope Affected paths and dependency versions	**Origin** Source run, commit, issue URL, creation time
Executable evidence Frozen regression source and SHA-256 hash	**Execution provenance** Original and candidate hashes, sandbox image, operation IDs
Reasoning boundaries Assumptions, rejected hypotheses, limitations	**Recall index** Optional summary, keywords, triggers, and embedding

Evidence gate: only an investigation with verdict `checks_passed`, a frozen regression source, and a matching regression hash can become active memory. Historical bulk import applies the same rule.

Portable bundle

`issue2proof.memory-bundle.v1` packages up to 100 capsules with a SHA-256 digest for each capsule and a manifest digest. Imports revalidate the schema, provenance, content digest, and regression hash atomically.

These hashes provide content integrity, not cryptographic authorship. Organization signing remains a possible extension. A portable bundle contains regression source; treat it like test code. The authenticated context endpoint can withhold that source from an editing agent.

Recall is routing. Replay is verification.

1 · Recall

Lexical: free token overlap over claims, paths, assumptions, boundaries, keywords, and triggers.

Semantic: one budgeted Qwen embedding batch ranks meaning rather than exact wording. Capsule vectors are cached.

Path-aware CI: changed paths boost capsules with matching scope.

2 · Replay

The current revision is imported and hashed. Selected frozen regressions run in fresh sandboxes.

`preserved`: valid evidence and exit code 0.

`violated`: valid assertion failure.

`uncertain`: setup or execution cannot cleanly decide.

PR title + changed files → rank relevant capsules → sandbox replay → merge verdict + evidence

Separation matters: high recall relevance does not prove a regression exists. It only decides which historical behaviors deserve execution. The sandbox result supplies the observed verdict.

Conflict analysis

If replay is violated or uncertain, Nemotron Super can explain the observed conflict. Nemotron Ultra is permitted only for an explicitly escalated high-risk case. Neither conflict-analysis stage silently edits the revision.

Different models have different jobs

Role	Model	Purpose
Fast indexing	`NVIDIA Nemotron-3 Nano 30B A3B`	Produces capsule keywords and future change triggers without rewriting evidence.
Engineering	`Nemotron-3 Super 120B A12B`	Creates regression and repair proposals; explains observed memory conflicts.
Deep reasoning	`Nemotron-3 Ultra 550B A55B`	Adjudicates only escalated, high-risk uncertainty and selects a next experiment.
Semantic retrieval	`Qwen3-Embedding-8B`	Embeds change descriptions and capsule text in one bounded batch.
Optional challenger	`Kimi-K2.7-Code`	Reserved for future disagreement experiments.

Applied AI concepts

Retrieval-augmented generation Relevant verified context is retrieved before an agent changes code.	Tool-using agents Models propose artifacts; controlled tools execute and observe them.
Generate–test–repair loop Candidate regressions and repairs pass discriminating experiments.	Model routing Task complexity and uncertainty select the smallest suitable model.
Structured outputs Strict schemas bound model responses and reject malformed output.	Long-term agent memory Durable evidence remains accessible across sessions and vendors.

What runs where, and what gets trusted?

Application host

- Validates and stores reports
- Orchestrates model calls
- Tracks jobs and budget
- Never executes repository Python

Nebius Sandbox

- Runs repository and generated code
- Uses disposable environments
- Disables network for test execution
- Records operation evidence

Execution controls

- Imports are bounded by archive size, entry count, expanded text, and per-file size.
- Unsafe paths, symlinks, encrypted archives, hidden credential files, and unsupported binaries are rejected.
- Dependency wheels are prepared separately with network access and no repository source.
- Tests run offline as uid 65534 with read-only inputs and memory, CPU, process, file, output, and wall-clock limits.
- Input hashes are checked after execution. Operation IDs and immutable image identifiers are recorded.

Authentication

A private deployment uses an application access token and signed session cookies. The Nebius API key stays server-side and is never the user login token.

Durability

Leased background jobs prevent duplicate paid work. Expired work becomes interrupted and requires explicit resume. Completed checkpoints, cancellation state, and cost reservations persist in SQLite.

Designed around a $25 credit limit

$25 Total configured credit	**$20**Spendable allowance	**$5**Protected reserve

How cost is controlled

- Lexical recall, portable-bundle recall, recorded examples, and local evaluation cost nothing.
- Semantic retrieval batches one query and all uncached capsules into one embedding request.
- Nano handles cheap indexing. Super handles normal engineering. Ultra requires an escalation signal and explicit allowance.
- Every job has model-call, sandbox-call, repair-attempt, and dollar reservation ceilings.
- Daily and lifetime ledgers stop new provider calls when the configured allowance is exhausted.

Practical hackathon strategy: judges can explore the public recorded-evidence interface without consuming credits. Fresh paid operations require explicit authorization in the authenticated backend.

Reservations are controller estimates and do not replace billing limits at the provider account level.

What the current numbers mean

30Blind Nemotron refactor trials	**8**Historical regressions introduced	**+5**Extra catches beyond plain CI

Check	Regressions detected	Detection within introduced regressions
Plain visible CI	3 of 8	37.5%
Capsule replay	8 of 8	100% in this focused experiment

The 30 trials cover two real public histories: Werkzeug safe path joining and Requests Unicode content length. The agent received repaired code and ordinary visible tests but did not receive the incident description or frozen capsule regression.

12/12 Correct lexical top-1 recall	**12/12**Correct path-aware top-1 recall

This separate routing evaluation uses 12 authored change descriptions across four verified histories.

Interpretation limits: the refactor benchmark is a focused two-case stress test, not an estimate for all software. Public cases may appear in model training data. Recall queries were authored from known incidents. Product audits use Nebius Sandboxes, while benchmark throughput tests ran locally. Every trial and limitation is included in the repository artifacts.

What Issue2Proof proves—and what it does not

A valid result supports

- The selected test executed in the recorded environment.
- The inputs matched their recorded hashes.
- The selected behavior passed, failed by assertion, or remained uncertain.
- The report preserves bounded provenance for review.

A valid result does not support

- Universal correctness of the entire repository.
- Freedom from every security vulnerability.
- Correctness outside selected memories and tests.
- Authorship authenticity from hashes alone.

Current product boundaries

- The investigation pipeline currently targets Python and pytest repositories.
- The public Vercel site is an interactive recorded-evidence demonstration. Fresh inference and sandbox operations belong in the authenticated FastAPI deployment.
- Portable bundles expose frozen regression source to their holder. Protected context recall withholds source when independent server-side replay is required.
- Capsule signing and organizational approval workflows are future extensions.

The honest claim: Issue2Proof makes selected historical behavior easier to find and independently replay. It provides stronger evidence than an unverified note, while remaining bounded by the tests and environment that were actually executed.

How to explain the project clearly

One sentence

Issue2Proof is regression memory for AI coding agents that survives the agent.

Thirty seconds

“Coding agents can already edit code and run tests. The missing layer is durable, portable evidence about failures that were fixed before. Issue2Proof turns a verified repair into a Memory Capsule containing the behavioral claim, provenance, hashes, and a frozen regression. When any future agent or pull request touches related code, it recalls that capsule and replays the behavior in a fresh Nebius Sandbox. In a focused 30-trial evaluation, plain CI caught 3 of 8 introduced regressions; capsule replay caught all 8.”

Three questions and answers

Question	Precise answer
Why is this different from Claude Code or Cursor?	Those tools remain the coding interface. Issue2Proof supplies shared executable memory and independent replay across them.
Why not just add a test?	A test is essential. A capsule also preserves why it exists, where it applies, its assumptions and boundaries, hashes, environment, and execution provenance, then routes it to relevant future changes.
Where is the AI value?	Models generate discriminating regressions and repairs, create retrieval indexes, semantically recall relevant evidence, and analyze conflicts. Execution—not model confidence—decides the recorded gate.

Quick reference

Glossary and project links

Memory Capsule A versioned behavioral contract linked to executable evidence and provenance.	**Frozen regression** The exact historical test source protected by its SHA-256 hash.
Recall Finding capsules relevant to a planned code change.	**Replay** Executing selected frozen regressions on a proposed revision.
Withheld check An existing test hidden from the repair model and used as an independent gate.	**Sandbox** A disposable isolated environment for untrusted or model-generated code.

Try and inspect

Public application
https://issue2proof-demo.vercel.app

Source repository
https://github.com/kushalar03/issue2proof

Integration guide
github.com/kushalar03/issue2proof/blob/main/docs/INTEGRATIONS.md

GitHub Action guide
github.com/kushalar03/issue2proof/blob/main/docs/GITHUB_ACTION.md

Final mental model: Git records what changed. An agent proposes what to change next. Issue2Proof remembers which previously broken behaviors must be checked again—and carries the executable evidence needed to check them.